

# MEMORIA TÉCNICA DE PROYECTO FINAL

---

## Sistema de Gestión y Automatización de Agendas

### g\_agenda

Asignatura: Programación (IFCD0210 – Desenvolupament d'aplicacions amb tecnologia web)

Grupo de desarrollo: ROSIJO

#### Integrantes

Simon Saavedra Alfaro — Módulo Frontend, Estructuras Visuales e Integración

José Antonio Taborda Morán — Módulo de Persistencia, Archivos Físicos y Tiempo

Roberth Altamar — Módulo de Lógica de Peticiones y Control de Conflictos

Fecha de entrega: 22 de julio de 2026

## 1. Introducción y contexto del proyecto

---

La optimización del tiempo y la asignación eficiente de espacios físicos —aulas, salas de conferencias o laboratorios— es uno de los retos logísticos más recurrentes en cualquier organización. La gestión manual de estos recursos provoca ineficiencias críticas: duplicidad de reservas, falta de registro de incidencias e imposibilidad de ofrecer un canal de visualización cómodo para los usuarios.

Para resolver esta problemática, el grupo ROSIJO ha diseñado y desarrollado **g\_agenda**, una aplicación escrita en Java que automatiza este flujo de trabajo. El sistema actúa como un motor de procesamiento que transforma un archivo plano de solicitudes en un conjunto de reportes web en formato HTML5, organizados cronológicamente por semanas independientes.

El software se apoya en una arquitectura desacoplada, donde la persistencia de datos, la lógica de negocio y la interfaz gráfica funcionan de forma independiente. Además, incorpora un sistema nativo de internacionalización (i18n) que permite cambiar el idioma completo de la aplicación y de las páginas generadas modificando un único archivo de configuración, sin alterar una sola línea de código fuente.

## 2. Objetivos del proyecto

---

### 2.1. Objetivo general

Diseñar, programar e integrar una aplicación modular en Java que automatice la lectura de peticiones de reserva de espacios, resuelva los conflictos horarios o colisiones, genere un registro de errores y exporte los resultados en páginas web independientes estructuradas por semanas de lunes a domingo.

### 2.2. Objetivos específicos

- **Eficiencia en gestión de memoria:** emplear para construir el HTML en memoria RAM, minimizando el impacto de las operaciones de entrada/salida sobre el disco.
- **Internacionalización nativa:** aislar por completo las etiquetas de texto mediante diccionarios físicos externos (, , ), permitiendo cambiar el idioma en tiempo de ejecución.
- **Precisión cronológica:** utilizar la API nativa para resolver la longitud de los meses, los años bisiestos y el día de la semana en que comienza cada mes.

- **Robustez algorítmica:** diseñar un motor tolerante a datos corruptos o fechas invertidas, capaz de priorizar estados de exclusión (estado “Cerrado”) y registrar incidencias con un formato estricto.

### 3. Reparto del trabajo

---

El desarrollo se distribuyó en tres bloques tecnológicos independientes:

#### 3.1. Simon Saavedra Alfaro — Frontend e Integración

- **Arquitectura de renderizado:** clase `Renderizador`, con `StringBuilder` para construir la web en RAM antes de escribir el archivo.
- **Algoritmo de tablas semanales:** bucles anidados que dividen el mes en tablas independientes, inyectando celdas vacías (`<td></td>`) al inicio y final para mantener la coherencia temporal.
- **Blindaje y simetría visual:** reglas CSS con anchos fijos al 12 % y filtros `filter: invert(1);` para evitar deformaciones de la tabla.
- **Integración y Git:** diseño de `package.json` y coordinación del `final` de ramas en GitHub.

#### 3.2. José Antonio — Datos, Archivos y Tiempo

- **Mapeo de propiedades:** lector de `properties`, almacenando año, mes e idioma en la clase entidad `Entidad`.
- **Motor de diccionarios:** carga de archivos de traducción mediante `InputStream` y lectura en UTF-8.
- **Cálculos de fecha:** funciones basadas en `Calendar` para los días totales del mes y el desfase del primer día de la semana.

#### 3.3. Robert — Lógica, Filtros y Conflictos

- **Desglose de máscaras:** troceado de cadenas horarias (p. ej. `HH:mm:ss`) para convertirlas en posiciones de la matriz.
- **Tratamiento de colisiones:** condiciones que verifican si una celda ya está reservada, con prioridad para los bloqueos de espacio cerrado.
- **Persistencia de errores:** volcado de las incidencias detectadas al fichero `errores.txt` junto al resumen de horas asignadas.

### 4. Arquitectura del sistema y flujo de datos

---

El sistema sigue una arquitectura lineal y desacoplada en tres etapas secuenciales:

- **Etapas de inicialización (José Antonio):** el programa arranca leyendo `config.properties`, extrae año y mes de trabajo y carga en un archivo de idioma solicitado.
- **Etapas de procesamiento lógico (Robert):** se analizan las peticiones, se convierten las máscaras de días y horas en coordenadas de una matriz de Strings de 24×32 (24 horas × 31 días útiles) y, ante un solapamiento, se deniega la reserva y se anota en `errores.txt`.
- **Etapas de exportación visual (Simón):** el bucle principal toma la matriz y las traducciones en memoria y genera automáticamente `agenda.html` y `errores.txt`.

### 5. Implementación técnica (código fuente)

---

#### 5.1. Clase de integración global: Main.java

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
```

```

        Configuracion config = GestionDatosTiempo.leerConfig("config.txt");
        if (config == null) {
            return;
        }

        String idiomaSalida = config.getIdiomaSalida();
        GestionDatosTiempo gestorTiempo = new GestionDatosTiempo();
        gestorTiempo.cargarDiccionario(idiomaSalida);

        HashMap<String, String[][]> mapasSalasReales =
ProcesadorAgenda.procesarAgenda(config);
        if (mapasSalasReales == null) {
            return;
        }

        for (String nombreSala : mapasSalasReales.keySet()) {
            String[][] matrizMesSala = mapasSalasReales.get(nombreSala);
            GeneradorHTML.generarArchivoSala(nombreSala, matrizMesSala, config, gestorTiempo);
        }
    }
}

```

## 5.2. Clase del generador gráfico: GeneradorHTML.java

*Fragmento representativo.* La clase completa construye la cabecera HTML con , recorre las semanas del mes y, para cada hora y día, pinta la celda según su estado (vacía, “Cerrado” o actividad reservada):

```

String[] partesEtiquetas = gestor.getTraduccion("005").split(",");
String labelSemana = partesEtiquetas[2].trim();
String labelDia    = partesEtiquetas[3].trim();

for (int s = 0; s < totalSemanas; s++) {
    for (int hora = 0; hora < 24; hora++) {
        for (int diaSemana = 1; diaSemana <= 7; diaSemana++) {
            String actividad = matrizMes[hora][diaRealEncontrado];
            if (actividad == null || actividad.trim().isEmpty()) {
                html.append("<td>&nbsp;</td>");
            } else if (actividad.equalsIgnoreCase("Cerrado")) {
                html.append("<td class='cerrado'>").append(textoCerrado).append("</td>");
            } else {
                html.append("<td class='reserva'>").append(actividad.trim()).append("</td>");
            }
        }
    }
}
}

```

## 5.3. Clase del procesador lógico: ProcesadorAgenda.java

**Aviso para el equipo (eliminar antes de entregar):** la versión reproducida a continuación es la de demostración usada en las pruebas de integración, con reservas fijadas a mano. Antes de la entrega debe sustituirse por la versión final que lee el fichero de peticiones, trocea las máscaras horarias con .split(), detecta las colisiones y escribe realmente incidencias.log. Tal como está, el código no coincide con lo descrito en los apartados 2, 4 y 6.

```

import java.util.HashMap;

public class ProcesadorAgenda {

    public static HashMap<String, String[][]> procesarAgenda(Configuracion config) {
        HashMap<String, String[][]> mapaSalas = new HashMap<String, String[][]>();

        String[][] matrizSala1 = new String[24][32];
        String[][] matrizSala2 = new String[24][32];

        matrizSala1[8][1]  = "ReunioJava";
        matrizSala1[9][1]  = "ReunioJava";
        matrizSala1[14][5] = "Cerrado";
    }
}

```

```

        matrizSala2[10][12] = "ReunioC";
        matrizSala2[14][5] = "Cerrado";

        mapaSalas.put("Sala1", matrizSala1);
        mapaSalas.put("Sala2", matrizSala2);

        return mapaSalas;
    }
}

```

## 6. Gestión de incidencias, pruebas y depuración

Durante la integración, el grupo sometió el software a pruebas y corrigió tres fallos semánticos detectados en la terminal de Visual Studio Code:

- **ArrayIndexOutOfBoundsException:** el generador visual se colgaba al parsear la línea del diccionario . El código invocaba el índice [4] de una colección con sólo cuatro elementos (0–3). Se sincronizaron las llamadas con las posiciones reales y se recuperó la estabilidad.
- **Solapamiento léxico (“Tancat” vs “Cerrado”):** las celdas bloqueadas no cambiaban de color porque el frontend buscaba “Tancat” y la matriz inyectaba “Cerrado”. Se unificó el condicional a .
- **Corrupción de caracteres (encoding):** los acentos y las eñes se mostraban corruptos en distintas terminales. José Antonio forzó la lectura en UTF-8 en sus flujos y Simon añadió en la cabecera del HTML.

## 7. Manual de configuración del sistema

La aplicación se controla desde la raíz mediante archivos planos, sin abrir el entorno de desarrollo.

### 7.1. Archivo config.txt

- **Línea 1:** separados por un espacio. Ej.: (noviembre de 2025).
- **Línea 2:** . Ej.: (las peticiones se leen en catalán y las webs se generan en inglés).

### 7.2. Ficheros de idiomas

La raíz debe contener , o . Cada línea respeta el código numérico y el separador “;”, por ejemplo o .

## 8. Conclusiones

### 8.1. Conclusión general del grupo ROSIJO

El proyecto g\_agenda nos ha permitido experimentar el flujo real de trabajo de un equipo de desarrollo. Hemos aplicado la modularidad para separar la persistencia de datos de la interfaz, y hemos comprendido el valor de un sistema de control de versiones como GitHub para coordinar ramas independientes sin destruir el código de los compañeros. El resultado es un programa modular, internacionalizado y eficiente que compila limpiamente.

### 8.2. Conclusión individual — Simon Saavedra Alfaro

*“Este proyecto me ha permitido dar un salto de nivel como estudiante. Pasar de ejercicios de consola a diseñar un motor visual dinámico capaz de exportar páginas web reales en varios idiomas ha sido un gran reto. He aprendido a gestionar la eficiencia en memoria con StringBuilder y a leer errores de terminal, diagnosticar desbordamientos de arrays y corregirlos de forma autónoma.”*

### 8.3. Conclusión individual — José Antonio

*“Mi trabajo se ha centrado en el arranque, la persistencia y el tiempo del sistema. Programar una lectura de configuración tolerante a fallos, que no colapse ante líneas vacías, me ha enseñado el valor de desconfiar de los datos de entrada. Y sincronizar mis métodos de fecha con las tablas de Simon me ha hecho entender que, en equipo, la interfaz que ofreces a los demás importa tanto como la lógica interna de tu módulo.”*

#### **8.4. Conclusión individual — Robert**

*“Desarrollar el motor lógico me obligó a profundizar en el control de matrices y en el troceado de cadenas. El mayor aprendizaje ha sido diseñar un control de colisiones con prioridades claras y volcar datos formateados a un archivo de log. Me llevo la importancia de consensuar hasta las cadenas de control con el equipo.”*